Operating Systems ? Midterm Study Notes

1. Processes and Threads

A process is a program in execution. It owns an address space, a program counter, a stack, a heap, and a set of open file descriptors. The operating system keeps a Process Control Block (PCB) for every process. The PCB stores the process state, program counter, CPU registers, scheduling information, memory-management information, and accounting data.

Process states in the classic five-state model:

1. New ? the process is being created.
2. Ready ? the process is waiting to be assigned to a CPU.
3. Running ? instructions are being executed.
4. Waiting (blocked) ? the process is waiting for an event such as I/O completion.
5. Terminated ? the process has finished execution.

A context switch saves the state of the current process in its PCB and loads the state of the next process. Context switches are pure overhead: no useful user work happens during the switch, and the cost grows with the number of registers and cache footprint.

A thread is the smallest unit of execution scheduled by the CPU. Threads in the same process share the code section, data section, and open files, but each thread has its own program counter, register set, and stack. Benefits of threads include responsiveness, resource sharing, lower creation cost than processes, and better scalability on multicore CPUs.

User-level threads are managed by a library without kernel involvement, so switching is fast, but one blocking system call can block the entire process. Kernel-level threads are scheduled by the kernel, so a blocking call only blocks one thread, but creation and switching are more expensive.

2. CPU Scheduling

The scheduler decides which ready process runs next. Performance metrics include CPU utilization, throughput, turnaround time (completion minus arrival), waiting time (time spent in the ready queue), and response time (time from submission to first response).

First-Come, First-Served (FCFS) schedules in arrival order. It is simple but suffers from the convoy effect: one long job delays every short job behind it.

Shortest Job First (SJF) picks the process with the smallest next CPU burst. It is optimal for average waiting time among non-preemptive algorithms, but the next burst length is not knowable in advance, so it is usually approximated by exponential averaging of past bursts. Shortest Remaining Time First (SRTF) is the preemptive version.

Round Robin (RR) gives each process a fixed time quantum. If the process does not finish within the quantum, it is preempted and placed at the tail of the ready queue. RR provides good response time, but a very small quantum makes context-switch overhead dominate, while a very large quantum degenerates into FCFS.

Priority scheduling runs the highest-priority process first. It can starve low-priority processes; aging solves this by gradually increasing the priority of waiting processes.

Multilevel feedback queues use several queues with different priorities and quanta, and move processes between queues based on observed behavior. They approximate SJF without knowing burst lengths in advance.

Real-time scheduling includes Rate Monotonic (RM), which assigns higher priority to shorter periods, and Earliest Deadline First (EDF), which is optimal for dynamic priorities when total utilization is at most 1. RM is optimal among fixed-priority algorithms.

3. Synchronization

A race condition occurs when multiple processes access shared data concurrently and the final result depends on the interleaving. The critical section is the code that accesses shared data. A correct solution must satisfy three properties:

- Mutual exclusion ? at most one process is inside the critical section.
- Progress ? if the critical section is free, some waiting process must be able to enter.
- Bounded waiting ? there is a limit on how many times other processes can enter before a waiting process gets its turn.

Peterson's solution is a software solution for two processes using two shared variables, flag and turn. It satisfies all three requirements but relies on sequential consistency and does not scale to many processes.

A mutex lock provides mutual exclusion with acquire() and release(). A spinlock busy-waits; it is useful on multiprocessors for very short critical sections but wastes CPU on a single core.

A semaphore is an integer with two atomic operations: wait() (also called P) decrements and blocks if the value is negative, and signal() (V) increments and wakes a waiting process. A binary semaphore behaves like a mutex. A counting semaphore controls access to a finite pool of resources.

Monitors combine mutual exclusion with condition variables. wait on a condition variable releases the monitor lock and blocks; signal wakes one waiter. Monitors are easier to get right than raw semaphores because the compiler enforces the lock.

Classic problems:

- Producer?Consumer uses a bounded buffer and two semaphores: empty (initially N) and full (initially 0), plus a mutex for the buffer.
- Readers?Writers allows multiple readers or one writer. The first readers?writers solution can starve writers.
- Dining Philosophers illustrates deadlock and starvation when all five philosophers pick up their left fork simultaneously. Solutions include limiting diners to four, picking up forks in a global order, or using a monitor with state.

4. Deadlock

A deadlock requires all four Coffman conditions simultaneously:

1. Mutual exclusion ? a resource is held in a non-shareable mode.
2. Hold and wait ? a process holds one resource while waiting for another.
3. No preemption ? resources cannot be forcibly taken away.
4. Circular wait ? a cycle of processes each waiting for a resource held by the next.

Deadlock handling strategies:

- Prevention breaks one of the four conditions, for example by requiring all resources to be requested at once (breaks hold and wait) or by imposing a total ordering on resources (breaks circular wait).
- Avoidance uses advance knowledge of maximum claims. The Banker's algorithm checks whether granting a request leaves the system in a safe state, meaning there exists at least one sequence in which every process can finish. If no safe sequence exists, the request is denied.
- Detection and recovery builds a wait-for graph; a cycle indicates deadlock. Recovery can terminate processes or preempt resources.
- Ignoring the problem is the practical choice in many general-purpose systems (the Ostrich algorithm).

A safe state is not the same as a deadlocked state: a safe state guarantees that some completion order exists, while an unsafe state may still avoid deadlock if processes behave favorably. The Banker's algorithm can only avoid unsafe states; it cannot prevent a process from eventually requesting its declared maximum.

5. Memory Management

Logical addresses are generated by the CPU; physical addresses refer to actual memory. The Memory Management Unit (MMU) translates between them.

Paging divides logical memory into fixed-size pages and physical memory into frames. A page table maps page numbers to frame numbers. The Translation Lookaside Buffer (TLB) caches recent translations; a TLB hit avoids an extra memory access. Effective access time is hit_ratio × (TLB + memory) + (1 ? hit_ratio) × (TLB + 2 × memory).

A page fault occurs when the referenced page is not in memory. The OS finds a free frame or evicts a page, loads the page from disk, updates the page table, and restarts the instruction.

Page replacement algorithms:

- FIFO evicts the oldest page. It can suffer Belady's anomaly, where adding frames increases the fault rate.
- Optimal (OPT) evicts the page used farthest in the future. It is a theoretical benchmark, not implementable online.
- LRU evicts the least recently used page and approximates OPT well, but exact LRU requires hardware support or costly bookkeeping.
- Clock (second-chance) approximates LRU with a reference bit.

Thrashing happens when the sum of working sets exceeds physical memory, so the system spends most of its time paging. Solutions include working-set modeling and page-fault frequency control.

Segmentation divides memory into variable-length logical segments such as code, stack, and heap. It matches programmer view but causes external fragmentation. Paging avoids external fragmentation but can cause internal fragmentation in the last page.

Virtual memory lets a process execute with only part of its address space resident, enabling demand paging, shared libraries, and copy-on-write fork.

6. File Systems

A file is a named collection of related information. File attributes include name, identifier, type, location, size, protection, and timestamps. Access methods include sequential, direct, and indexed access.

Contiguous allocation gives each file a contiguous block range: fast sequential and direct access, but external fragmentation and difficult growth. Linked allocation stores each block with a pointer to the next: no external fragmentation, but random access is slow. Indexed allocation keeps an index block of pointers: good random access, at the cost of index-block overhead.

A directory maps file names to file control blocks. Inodes store metadata and block pointers; the file name lives in the directory entry.

Journaling writes metadata changes to a log before applying them, so a crash can be recovered by replaying or discarding the log. Copy-on-write file systems never overwrite live blocks, which makes snapshots cheap.

Disk scheduling algorithms include FCFS, SSTF (shortest seek time first, which can starve far requests), SCAN (elevator algorithm), C-SCAN, and LOOK. RAID 0 stripes for performance; RAID 1 mirrors for reliability; RAID 5 stripes with distributed parity and tolerates one disk failure; RAID 6 uses two parity blocks and tolerates two failures.